Plan de Gestión y Desarrollo Integral del Sistema

> **Proyecto:** Sistema de Gestión y Disposición de Equipos Electrónicos (RAEE) y Ventas
> **Metodología de Gestión:** Estándar PMI (PMBOK) + Marco Ágil Scrum
> **Tecnología Base:** Java 21, Spring Boot 3.3.4, Spring Data JPA, Hibernate, MySQL 8 / H2 Database

---

1. Plan de Proyecto según las Etapas de Gestión Tradicional (PMI)

El proyecto se estructura bajo el ciclo de vida de los 5 grupos de procesos establecidos por el **Project Management Institute (PMI - PMBOK)**:

```
flowchart LR
    A["1. Inicio<br/>(Project Charter)"] --> B["2. Planificación<br/>(WBS / Cronograma / Riesgos)"]
    B --> C["3. Ejecución<br/>(Desarrollo Spring Boot / JPA)"]
    C --> D["4. Monitoreo y Control<br/>(Testing / Calidad / Auditoría)"]
    D --> E["5. Cierre<br/>(Entrega / Lecciones Aprendidas)"]
    D -. Feedback .-> B
```

1.1. Etapa de Inicio (Initiating)

- **Acta de Constitución (Project Charter):**
  - **Propósito:** Automatizar el control de inventario de equipos tecnológicos en desuso o reciclables (RAEE), garantizando la trazabilidad de categorías, clientes y ventas/disposiciones con alta integridad transaccional.
  - **Objetivos SMART:**
    1. Implementar un backend en Spring Boot con arquitectura en capas en un plazo de 4 semanas.
    2. Desarrollar un modelo relacional normalizado en 3FN con 5 entidades interconectadas.
    3. Lograr una cobertura de pruebas automatizadas superior al 90% en la capa de servicios y persistencia.
  - **Interesados Clave (Stakeholders):**
    - *Product Owner / Patrocinador:* Define prioridades de negocio y criterios de aceptación.
    - *Líder de Proyecto / Scrum Master:* Gestiona cronograma, riesgos y facilitación técnica.
    - *Equipo de Desarrollo Backend (Java/JPA):* Diseña e implementa la persistencia, servicios y APIs REST.
    - *Equipo de Aseguramiento de la Calidad (QA):* Valida cumplimiento de requerimientos y pruebas de carga/seguridad.

1.2. Etapa de Planificación (Planning)

- **Plan de Gestión del Alcance (EDT / WBS):**

- **Nivel 1:** Sistema de Gestión RAEE y Ventas
  - **1.1 Arquitectura y Base de Datos:** Modelo ER, DDL SQL, Conexión Spring Boot (MySQL / H2).
  - **1.2 Persistencia JPA:** Entidades, Mapeo relacional, `@NamedQueries`, `JpaRepository`.
  - **1.3 Lógica de Negocio y Servicios:** Validaciones transaccionales, consultas JPQL seguras.
  - **1.4 Controladores y Exposición API:** Endpoints CRUD, códigos de respuesta HTTP (`RESTful`).
  - **1.5 Verificación y Testing:** Pruebas unitarias (Mockito), pruebas de integración (`@SpringBootTest`).

- **Cronograma e Hitos Principales:**

Hito	Entregable Clave	Plazo Estimado
H1	Acta de Constitución, Requerimientos y Diseño DER aprobado	Semana 1
H2	Sprint 1 Finalizado: Configuración BD, Entidades y CRUDs Básicos	Semana 2
H3	Sprint 2 Finalizado: Módulo de Ventas, JPQL seguro, Named Queries y Frontend	Semana 3
H4	Pruebas integrales de aceptación, informe final y Cierre formal	Semana 4

- **Matriz de Gestión de Riesgos:**

ID	Riesgo Identificado	Prob.	Impacto	Estrategia de Mitigación
R1	Indisponibilidad o fallo del servidor local MySQL (XAMPP).	Media	Alto	Implementar perfil dual con H2 en memoria (`application-h2.properties`) para ejecución autónoma.
R2	Inyección SQL/JPQL en consultas personalizadas con filtros dinámicos.	Baja	Crítico	Utilizar obligatoriamente parámetros seguros (`:param`) con `@Param` de Spring Data JPA.
R3	Inconsistencias por concurrencia en stock durante transacciones de venta.	Media	Alto	Uso de anotaciones transaccionales `@Transactional` y restricciones `NOT NULL` / `CHECK` en BD.

1.3. Etapa de Ejecución (Executing)

- Coordinación del equipo de desarrollo, implementación de código fuente siguiendo principios SOLID y patrones de diseño (Repository, DTO, Controller-Service-Repository).
- Configuración de la base de datos MySQL 8 e integración continua con Maven Wrapper.

1.4. Etapa de Monitoreo y Control (Monitoring & Controlling)

- Revisiones de código (*Code Reviews*) orientadas a la eficiencia de consultas JPA (evitar problema de *N+1 SELECTs*).
- Verificación continua de pruebas automatizadas mediante `mvnw test`.
- Monitoreo de logs Hibernate (`show-sql=true`) para auditar la generación de sentencias DDL y DML.

1.5. Etapa de Cierre (Closing)

- Verificación de la Lista de Criterios de Aceptación con el usuario final.
- Elaboración del documento de lecciones aprendidas (beneficios de persistencia polimórfica y dialectos agnósticos).
- Entrega del repositorio, scripts DDL y colección de pruebas en Postman.

---

2. Product Backlog del Sistema

El Backlog agrupa las necesidades del negocio priorizadas según el método **MoSCoW** (*Must Have*, *Should Have*, *Could Have*, *Won't Have*) y estimadas en Story Points (Fibonacci):

ID	Épica	Historia de Usuario (HU)	Story Points	Prioridad MoSCoW
HU-01	Categorías	Como administrador, quiero registrar y clasificar categorías de productos para ordenar el inventario.	3	**Must Have**
HU-02	Clientes	Como vendedor, quiero registrar y gestionar clientes con su DNI/RUC para asociar las operaciones comerciales.	5	**Must Have**
HU-03	Productos	Como almacenero, quiero registrar, listar, editar y eliminar equipos tecnológicos para mantener el inventario al día.	8	**Must Have**
HU-04	Ventas	Como cajero, quiero registrar una venta asignada a un cliente con múltiples ítems de productos para procesar la salida.	13	**Must Have**
HU-05	Consultas	Como supervisor, quiero filtrar productos por estado operativo y categoría mediante JPQL para emitir reportes.	5	**Should Have**
HU-06	Named Queries	Como auditor, quiero consultar clientes por su número de identificación exacto usando parámetros seguros para prevenir fraudes.	3	**Should Have**
HU-07	Dashboard Web	Como usuario final, quiero una interfaz web limpia para visualizar y probar el CRUD directamente sin herramientas externas.	5	**Could Have**
HU-08	Exportación	Como administrador, quiero exportar las transacciones de ventas en formato PDF/Excel.	8	**Won't Have (Fase 2)**

---

3. Planificación de Sprints y Criterios de Aceptación

```
flowchart TD
    subgraph Sprint 1 : Base Arquitectónica y Catálogos
        S1A["HU-01: Gestión de Categorías"]
        S1B["HU-02: CRUD de Clientes"]
        S1C["HU-03: CRUD de Productos"]
        S1D["Configuración de Persistencia JPA y Perfiles BD"]
    end
    subgraph Sprint 2 : Ventas, Consultas Seguras y Testing
        S2A["HU-04: Transacciones de Venta y Detalle"]
        S2B["HU-05: Consultas JPQL con JOIN"]
        S2C["HU-06: Implementación de @NamedQueries"]
        S2D["HU-07: Batería de Pruebas Unitarias e Integración"]
    end
    Sprint 1 --> Sprint 2
```

3.1. Sprint 1: Núcleo del Modelo y Gestión Básica

- **Sprint Goal:** Establecer la infraestructura de datos, conexión Spring Boot y operaciones CRUD para Categorías, Clientes y Productos.
- **Duración:** 2 semanas (16 Story Points).
- **Historias Asignadas:** `HU-01`, `HU-02`, `HU-03`.
- **Criterios de Aceptación (Formato Gherkin):**
    - **Escenario 1 (Creación de Producto):**
        - *Dado que* el usuario proporciona datos válidos (nombre: "Laptop HP", precio: 1850.0, estado: "OPERATIVO").
        - *Cuando* se envía una petición `POST /api/productos`.
        - *Entonces* el sistema almacena el registro en la tabla `productos`, asigna un ID autoincremental y responde con código HTTP `201 Created`.
    - **Escenario 2 (Validación de duplicados en Clientes):**
        - *Dado que* existe un cliente con número de identificación `"72345678"`.
        - *Cuando* se intente registrar otro cliente con el mismo documento.
        - *Entonces* la capa JPA debe rechazar la inserción respetando la restricción de unicidad (`unique = true`).

3.2. Sprint 2: Ventas, Consultas Avanzadas y Aceptación

- **Sprint Goal:** Implementar el módulo transaccional de ventas, consultas JPQL y Named Queries con parámetros seguros y batería integral de pruebas.
- **Duración:** 2 semanas (21 Story Points).
- **Historias Asignadas:** `HU-04`, `HU-05`, `HU-06`, `HU-07`.
- **Criterios de Aceptación:**
    - **Escenario 1 (Registro de Venta en Cascada):**
        - *Dado que* un cliente seleccionado adquiere 2 unidades de un producto.

        - *Cuando* se envía la venta con su lista de detalles mediante `POST /api/ventas`.
        - *Entonces* la venta y los detalles deben persistirse atómicamente (`CascadeType.ALL`) y el total debe ser consistente con la suma de subtotales.
    - **Escenario 2 (Inmunidad a Inyección SQL/JPQL):**
        - *Dado que* se ejecutan filtros por estado o identificación de cliente con caracteres especiales de SQL.
        - *Cuando* el repositorio procesa la consulta con `@Param("param")`.
        - *Entonces* Hibernate procesa el parámetro como un valor literal y no altera la estructura de la consulta.

---

4. Diseño del Modelo Entidad-Relación (MER / DER)

4.1. Diagrama Entidad-Relación (Mermaid)

```
erDiagram
    CATEGORIAS ||--o{ PRODUCTOS : "clasifica"
    CLIENTES ||--o{ VENTAS : "realiza"
    VENTAS ||--|{ DETALLE_VENTAS : "contiene"
    PRODUCTOS ||--o{ DETALLE_VENTAS : "se_incluye_en"

    CATEGORIAS {
        BIGINT id PK
        VARCHAR nombre UK
        VARCHAR descripcion
    }

    PRODUCTOS {
        BIGINT id PK
        VARCHAR nombre
        VARCHAR marca
        VARCHAR modelo
        VARCHAR estado
        DECIMAL precio
        INT stock
        DATETIME fecha_registro
        BIGINT categoria_id FK
    }

    CLIENTES {
        BIGINT id PK
        VARCHAR nombre
        VARCHAR identificacion UK
        VARCHAR email
        VARCHAR telefono
        VARCHAR tipo
    }

    VENTAS {
        BIGINT id PK
        DATETIME fecha_venta
        DECIMAL total
        VARCHAR estado
        BIGINT cliente_id FK
    }

    DETALLE_VENTAS {
        BIGINT id PK
        BIGINT venta_id FK
        BIGINT producto_id FK
        INT cantidad
        DECIMAL precio_unitario
        DECIMAL subtotal
    }
```

4.2. Diccionario de Datos y Cardinalidades

1. **CATEGORIAS (1) a PRODUCTOS (N):** Una categoría clasifica a 0 o muchos productos; cada producto pertenece estrictamente a una categoría.
2. **CLIENTES (1) a VENTAS (N):** Un cliente puede registrar múltiples ventas a lo largo del tiempo.
3. **VENTAS (1) a DETALLE_VENTAS (N):** Cada venta se compone de uno o más ítems de detalle. Posee ciclo de vida compuesto (*Composición* en UML / `CascadeType.ALL` y `orphanRemoval = true`).
4. **PRODUCTOS (1) a DETALLE_VENTAS (N):** Un producto puede figurar en varios detalles de venta.

---

5. Creación de la Base de Datos y Conexión Spring Boot

5.1. Script DDL para MySQL (schema.sql)

El script crea la base de datos `raee_db` e inicializa las llaves primarias, foráneas e índices de integridad:

```sql
CREATE DATABASE IF NOT EXISTS `raee_db` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE `raee_db`;

CREATE TABLE IF NOT EXISTS `categorias` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `nombre` VARCHAR(255) NOT NULL UNIQUE,
    `descripcion` VARCHAR(255)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `clientes` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `nombre` VARCHAR(255) NOT NULL,
    `identificacion` VARCHAR(50) UNIQUE,
    `email` VARCHAR(100),
    `telefono` VARCHAR(50),
    `tipo` VARCHAR(50)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `productos` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `nombre` VARCHAR(255) NOT NULL,
    `marca` VARCHAR(255),
    `modelo` VARCHAR(255),
    `estado` VARCHAR(50) DEFAULT 'RECIBIDO',
    `precio` DECIMAL(10,2) DEFAULT 0.00,
    `stock` INT DEFAULT 1,
    `fecha_registro` DATETIME DEFAULT CURRENT_TIMESTAMP,
    `categoria_id` BIGINT,
    CONSTRAINT `fk_producto_categoria` FOREIGN KEY (`categoria_id`) REFERENCES `categorias`(`id`) ON DEL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `ventas` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `fecha_venta` DATETIME DEFAULT CURRENT_TIMESTAMP,
    `total` DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    `estado` VARCHAR(50) DEFAULT 'EMITIDA',
    `cliente_id` BIGINT NOT NULL,
    CONSTRAINT `fk_venta_cliente` FOREIGN KEY (`cliente_id`) REFERENCES `clientes`(`id`) ON DELETE RESTR
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `detalle_ventas` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `venta_id` BIGINT NOT NULL,
    `producto_id` BIGINT NOT NULL,
    `cantidad` INT NOT NULL,
    `precio_unitario` DECIMAL(10,2) NOT NULL,
    `subtotal` DECIMAL(10,2) NOT NULL,
    CONSTRAINT `fk_detalle_venta` FOREIGN KEY (`venta_id`) REFERENCES `ventas`(`id`) ON DELETE CASCADE,
    CONSTRAINT `fk_detalle_producto` FOREIGN KEY (`producto_id`) REFERENCES `productos`(`id`) ON DELETE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2. Configuración en Spring Boot

- **Perfil MySQL (application.properties):**

```
spring.application.name=upn
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/raee_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetr
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

- **Perfil H2 en Memoria (application-h2.properties):**

```
spring.application.name=upn
server.port=8080

spring.datasource.url=jdbc:h2:mem:raee_db;MODE=MySQL;DATABASE_TO_LOWER=TRUE;DEFAULT_NULL_ORDERING=HIGH
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

---

6. Implementación de Entidades con Anotaciones JPA

Cada entidad Java utiliza las anotaciones de la especificación Jakarta Persistence (`jakarta.persistence.*`):

6.1. Entidad Producto (Producto.java)

Incluye `@NamedQueries` para búsquedas parametrizadas por estado/marca y rangos de precio:

```java
@Data
@Entity
@Table(name = "productos")
@NamedQueries({
    @NamedQuery(
        name = "Producto.buscarPorEstadoYMarca",
        query = "SELECT p FROM Producto p WHERE p.estado = :estado AND p.marca = :marca"
    ),
    @NamedQuery(
        name = "Producto.buscarPorRangoPrecio",
        query = "SELECT p FROM Producto p WHERE p.precio BETWEEN :minPrecio AND :maxPrecio"
    )
})
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    private String marca;
    private String modelo;
    private String estado;
    private Double precio;
    private Integer stock;

    @Column(name = "fecha_registro")
    private LocalDateTime fechaRegistro;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "categoria_id")
    @JsonIgnoreProperties("productos")
    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    private Categoria categoria;

    @PrePersist
    public void prePersist() {
        if (this.fechaRegistro == null) this.fechaRegistro = LocalDateTime.now();
        if (this.estado == null || this.estado.trim().isEmpty()) this.estado = "RECIBIDO";
        if (this.precio == null) this.precio = 0.0;
        if (this.stock == null) this.stock = 1;
    }
}
```

6.2. Entidad Cliente (Cliente.java)

```java
@Data
@Entity
@Table(name = "clientes")
@NamedQueries({
    @NamedQuery(
        name = "Cliente.buscarPorIdentificacion",
        query = "SELECT c FROM Cliente c WHERE c.identificacion = :identificacion"
    ),
    @NamedQuery(
        name = "Cliente.buscarPorTipo",
        query = "SELECT c FROM Cliente c WHERE c.tipo = :tipo"
    )
})
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    @Column(unique = true)
    private String identificacion;

    private String email;
    private String telefono;
    private String tipo;
}
```

6.3. Entidades Venta y DetalleVenta (Venta.java / DetalleVenta.java)

```
@Data
@Entity
@Table(name = "ventas")
@NamedQueries({
    @NamedQuery(
        name = "Venta.buscarPorCliente",
        query = "SELECT v FROM Venta v WHERE v.cliente.id = :clienteId"
    ),
    @NamedQuery(
        name = "Venta.buscarPorEstado",
        query = "SELECT v FROM Venta v WHERE v.estado = :estado"
    )
})
public class Venta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "fecha_venta")
    private LocalDateTime fechaVenta;

    @Column(nullable = false)
    private Double total;

    private String estado;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "cliente_id", nullable = false)
    private Cliente cliente;

    @OneToMany(mappedBy = "venta", cascade = CascadeType.ALL, orphanRemoval = true)
    @JsonIgnoreProperties("venta")
    private List<DetalleVenta> detalles = new ArrayList<>();
}
```

---

7. Repositorios con JpaRepository

Al extender de `JpaRepository<T, ID>`, la aplicación hereda métodos estándar como `save()`, `findById()`, `findAll()`, `deleteById()` y `count()`, integrando a su vez las consultas declaradas:

- **ProductoRepository.java:**

```java
@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {

    @Query("SELECT p FROM Producto p WHERE p.estado = :estado")
    List<Producto> buscarPorEstado(@Param("estado") String estado);

    @Query("SELECT p FROM Producto p JOIN p.categoria c WHERE c.nombre = :nombreCat")
    List<Producto> buscarPorCategoria(@Param("nombreCat") String nombreCat);

    List<Producto> buscarPorEstadoYMarca(@Param("estado") String estado, @Param("marca") String marca);

    List<Producto> buscarPorRangoPrecio(@Param("minPrecio") Double minPrecio, @Param("maxPrecio") Double
}
```

- **ClienteRepository.java**:

```java
@Repository
public interface ClienteRepository extends JpaRepository<Cliente, Long> {

    Optional<Cliente> buscarPorIdentificacion(@Param("identificacion") String identificacion);

    List<Cliente> buscarPorTipo(@Param("tipo") String tipo);

    @Query("SELECT c FROM Cliente c WHERE LOWER(c.email) = LOWER(:email)")
    Optional<Cliente> buscarPorEmail(@Param("email") String email);
}
```

- **VentaRepository.java**:

```java
@Repository
public interface VentaRepository extends JpaRepository<Venta, Long> {

    List<Venta> buscarPorCliente(@Param("clienteId") Long clienteId);

    List<Venta> buscarPorEstado(@Param("estado") String estado);

    @Query("SELECT v FROM Venta v WHERE v.fechaVenta BETWEEN :inicio AND :fin")
    List<Venta> buscarPorRangoFechas(@Param("inicio") LocalDateTime inicio, @Param("fin") LocalDateTime
}
```

---

8. Implementación de Operaciones CRUD para Productos y Clientes

8.1. Endpoints REST Implementados

Módulo	Método HTTP	Ruta Endpoint	Descripción	Código Éxito
Productos	`GET`	`/api/productos`	Listar todos los productos	`200 OK`
Productos	`GET`	`/api/productos/{id}`	Buscar producto por ID	`200 OK` / `404 Not Found`
Productos	`POST`	`/api/productos`	Registrar nuevo producto	`201 Created`
Productos	`PUT`	`/api/productos/{id}`	Actualizar datos de un producto	`200 OK` / `404 Not Found`
Productos	`DELETE`	`/api/productos/{id}`	Eliminar producto por ID	`204 No Content` / `404 Not Found`
Clientes	`GET`	`/api/clientes`	Listar todos los clientes	`200 OK`
Clientes	`GET`	`/api/clientes/{id}`	Buscar cliente por ID	`200 OK` / `404 Not Found`
Clientes	`POST`	`/api/clientes`	Registrar nuevo cliente	`201 Created`
Clientes	`PUT`	`/api/clientes/{id}`	Actualizar cliente	`200 OK` / `404 Not Found`
Clientes	`DELETE`	`/api/clientes/{id}`	Eliminar cliente	`204 No Content` / `404 Not Found`

8.2. Capa de Controladores REST (ProductoController.java)

```java
@RestController
@RequestMapping("/api/productos")
public class ProductoController {

    @Autowired
    private ProductoService service;

    @GetMapping
    public List<Producto> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
        Producto p = service.buscarPorId(id);
        return p != null ? ResponseEntity.ok(p) : ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Producto> guardar(@RequestBody Producto producto) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.guardar(producto));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Producto> actualizar(@PathVariable Long id, @RequestBody Producto producto) {
        if (service.buscarPorId(id) == null) return ResponseEntity.notFound().build();
        producto.setId(id);
        return ResponseEntity.ok(service.guardar(producto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        if (service.buscarPorId(id) == null) return ResponseEntity.notFound().build();
        service.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}
```

---

9. Consultas JPQL y Named Queries con Parámetros Seguros

> [!IMPORTANT] **Seguridad y Prevención de Inyecciones SQL/JPQL:** La concatenación directa de cadenas en consultas JPA representa una vulnerabilidad crítica (CWE-89). Para mitigarlo, se utiliza enlace de variables con nombre (`:param`) vinculado mediante la anotación `@Param("param")` , lo que obliga al motor de base de datos a precompilar la sentencia (*Prepared Statements*) y tratar los argumentos exclusivamente como literales.

9.1. Consultas JPQL Directas

1. **Filtro de Productos por Estado:**

```
SELECT p FROM Producto p WHERE p.estado = :estado
```

2. **JOIN explícito entre Entidades (Producto y Categoría):**

```
SELECT p FROM Producto p JOIN p.categoria c WHERE c.nombre = :nombreCat
```

3. **Búsqueda insensible a mayúsculas de Cliente por Email:**

```
SELECT c FROM Cliente c WHERE LOWER(c.email) = LOWER(:email)
```

9.2. Named Queries declaradas en las Entidades

- En Producto.java:
  - `Producto.buscarPorEstadoYMarca`: `SELECT p FROM Producto p WHERE p.estado = :estado AND p.marca = :marca`
  - `Producto.buscarPorRangoPrecio`: `SELECT p FROM Producto p WHERE p.precio BETWEEN :minPrecio AND :maxPrecio`
- En Cliente.java:
  - `Cliente.buscarPorIdentificacion`: `SELECT c FROM Cliente c WHERE c.identificacion = :identificacion`
  - `Cliente.buscarPorTipo`: `SELECT c FROM Cliente c WHERE c.tipo = :tipo`
- En Venta.java:
  - `Venta.buscarPorCliente`: `SELECT v FROM Venta v WHERE v.cliente.id = :clienteId`

---

10. Evidencias de Ejecución y Pruebas

Se implementaron y ejecutaron satisfactoriamente pruebas automatizadas unitarias e integrales en ProyectoCompletoIntegrationTest.java y mediante script PowerShell test_api.ps1.

10.1. Matriz de Resultados de Pruebas

Caso de Prueba	Operación	Datos de Entrada	Resultado Esperado	Resultado Obtenido	Estado
CP-01	**CREATE (Cliente)**	DNI: `"45892134"`, Nombre: `"Ana Paredes"`	Inserción exitosa con ID asignado	HTTP 201 / Entidad con ID persistido	**PASS**
CP-02	**READ (Named Query)**	Identificación: `"45892134"`	Búsqueda segura por parámetro	Retorna cliente exacto sin SQL injection	**PASS**
CP-03	**UPDATE (Cliente)**	ID: `1`, Teléfono: `"999888777"`	Registro actualizado en base de datos	Entidad con nuevo teléfono reflejado	**PASS**
CP-04	**DELETE (Cliente)**	ID existente	Eliminación y confirmación	HTTP 204 / `Optional.empty()`	**PASS**
CP-05	**CREATE (Producto)**	Nombre: `"Switch Cisco"`, Precio: `820.0`	Persistencia con fecha autogenerada	Registro persistido en `productos`	**PASS**
CP-06	**JPQL con JOIN**	Categoría: `"Servidores"`	Listado filtrado por entidad relacionada	Registros vinculados retornados	**PASS**
CP-07	**Named Query (Rango)**	Precio entre `4000.0` y `5000.0`	Productos en el intervalo de precio	Lista correcta de registros filtrados	**PASS**
CP-08	**TRANSACCIÓN (Venta)**	Cliente ID + 2 Detalles de Venta	Venta + Detalles guardados en cascada	Total consistente y Venta asociada	**PASS**

10.2. Registro de Salida de Maven Test (`mvnw test`)

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.demo.FinalApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- FinalApplicationTests
[INFO] Running com.example.demo.ProductoServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- ProductoServiceTest
[INFO] Running com.example.demo.ProyectoCompletoIntegrationTest
Hibernate: insert into clientes (email,identificacion,nombre,telefono,tipo,id) values (?,?,?,?,?,default
Hibernate: insert into productos (categoria_id,estado,fecha_registro,marca,modelo,nombre,precio,stock,id
Hibernate: insert into ventas (cliente_id,estado,fecha_venta,total,id) values (?,?,?,?,default)
Hibernate: insert into detalle_ventas (cantidad,precio_unitario,producto_id,subtotal,venta_id,id) values
Hibernate: select v1_0.id,v1_0.cliente_id,v1_0.estado,v1_0.fecha_venta,v1_0.total from ventas v1_0 where
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- ProyectoCompletoIntegrationTest
[INFO]
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```